

Rapport - SAE BD-Maths

3.1 Modèle relationnel

3.1.1 Création de la base

Schéma utilisé : une table Aeroport et une table Vol avec clés primaires et clés étrangères.

```
DROP TABLE Vol CASCADE CONSTRAINTS;
DROP TABLE Aeroport CASCADE CONSTRAINTS;

CREATE TABLE Aeroport (
  id_aero  NUMBER PRIMARY KEY,
  nom_aero VARCHAR(100) NOT NULL UNIQUE,
  ville    VARCHAR(100),
  pays     VARCHAR(100)
);

CREATE TABLE Vol (
  compagnie      VARCHAR(100),
  num_vol        NUMBER,
  date_heure_dep  TIMESTAMP,
  date_heure_arr  TIMESTAMP,
  id_aero_dep     NUMBER,
  terminal_dep    VARCHAR(20),
  id_aero_arr     NUMBER,
  terminal_arr    VARCHAR(20),
  CONSTRAINT pk_vol PRIMARY KEY (compagnie, num_vol, date_heure_dep),
  CONSTRAINT fk_aero_dep FOREIGN KEY (id_aero_dep) REFERENCES Aeroport(id_aero),
  CONSTRAINT fk_aero_arr FOREIGN KEY (id_aero_arr) REFERENCES Aeroport(id_aero)
);
```

3.1.2 Requêtes

(a) Villes accessibles en vol direct depuis Paris

Explication : on filtre les vols au départ de Paris et on récupère les villes d'arrivée.

```
SELECT DISTINCT A.ville
FROM Vol V
JOIN Aeroport D ON V.id_aero_dep = D.id_aero
JOIN Aeroport A ON V.id_aero_arr = A.id_aero
WHERE D.ville = 'Paris';
```

(b) Villes accessibles avec une correspondance

Explication : on enchaîne deux vols avec contrainte horaire.

```
SELECT DISTINCT A2.ville
FROM Vol V1
JOIN Aeroport D1 ON V1.id_aero_dep = D1.id_aero
JOIN Vol V2 ON V1.id_aero_arr = V2.id_aero_dep
JOIN Aeroport A2 ON V2.id_aero_arr = A2.id_aero
WHERE D1.ville = 'Paris'
      AND V2.date_heure_dep > V1.date_heure_arr;
```

(c) Villes accessibles avec deux correspondances

Explication : même principe, avec trois vols en chaîne.

```
SELECT DISTINCT A3.ville
FROM Vol V1
JOIN Aeroport D1 ON V1.id_aero_dep = D1.id_aero
JOIN Vol V2 ON V1.id_aero_arr = V2.id_aero_dep
JOIN Vol V3 ON V2.id_aero_arr = V3.id_aero_dep
JOIN Aeroport A3 ON V3.id_aero_arr = A3.id_aero
WHERE D1.ville = 'Paris'
      AND V2.date_heure_dep > V1.date_heure_arr
      AND V3.date_heure_dep > V2.date_heure_arr;
```

(d) Villes accessibles avec un nombre quelconque de correspondances

Explication : requête récursive (base = vol direct, récursion = extension du trajet si horaire valide).

```
WITH Trajets (id_aero_arr, ville_arr, heure_arr) AS (  
    SELECT V.id_aero_arr, A.ville, V.date_heure_arr  
    FROM Vol V  
    JOIN Aeroport D ON V.id_aero_dep = D.id_aero  
    JOIN Aeroport A ON V.id_aero_arr = A.id_aero  
    WHERE D.ville = 'Paris'  
    UNION ALL  
    SELECT V_Suivant.id_aero_arr, A_Suivant.ville, V_Suivant.date_heure_arr  
    FROM Trajets T  
    JOIN Vol V_Suivant ON T.id_aero_arr = V_Suivant.id_aero_dep  
    JOIN Aeroport A_Suivant ON V_Suivant.id_aero_arr = A_Suivant.id_aero  
    WHERE V_Suivant.date_heure_dep > T.heure_arr  
)  
SELECT DISTINCT ville_arr FROM Trajets;
```

3.2 Modèle objet-relationnel

3.2.1 Implémentation des types complexes

Explication :

- equipageT (objet) + equipageTabT (nested table)
- indiceT (objet avec méthode get_impact) + indiceList (varray de 3)
- VOL_OR regroupe attributs simples + collections.

```

DROP TABLE VOL_OR CASCADE CONSTRAINTS;
DROP TYPE equipageTabT FORCE;
DROP TYPE equipageT FORCE;
DROP TYPE indiceList FORCE;
DROP TYPE indiceT FORCE;

CREATE OR REPLACE TYPE equipageT AS OBJECT (
    nom VARCHAR(50),
    fonction VARCHAR(50)
);
/

CREATE TYPE equipageTabT AS TABLE OF equipageT;
/

CREATE OR REPLACE TYPE indiceT AS OBJECT (
    nom_indice VARCHAR(50),
    valeur NUMBER,
    poids NUMBER,
    MEMBER FUNCTION get_impact RETURN NUMBER
);
/

CREATE OR REPLACE TYPE BODY indiceT AS
    MEMBER FUNCTION get_impact RETURN NUMBER IS
    BEGIN
        RETURN valeur * poids;
    END;
END;
/

CREATE TYPE indiceList AS VARRAY(3) OF indiceT;
/

CREATE TABLE VOL_OR (
    NumVol VARCHAR(20),
    AeroDep VARCHAR(20),
    DateHeureDep NUMBER,
    AeroArr VARCHAR(20),
    DateHeureArr NUMBER,
    Equipage equipageTabT,
    IndicesQualite indiceList
)
NESTED TABLE Equipage STORE AS Equipage_nt;

```

Jeu de données minimal :

```
INSERT INTO VOL_OR VALUES (
  'AF442', 'CDG', 1, 'GIG', 3,
  equipageTabT(
    equipageT('Goscinny', 'Pilote'),
    equipageT('Uderzo', 'Commissaire')
  ),
  indiceList(
    indiceT('carbone', 3, 4),
    indiceT('securite', 4, 5),
    indiceT('prix', 4, 3)
  )
);
COMMIT;
```

3.2.2 Requêtes

(a) Nombre de personnes d'équipage par fonction et par vol

```
SELECT v.NumVol, e.fonction, COUNT(*) AS nb_personnes
FROM VOL_OR v, TABLE(v.Equipage) e
GROUP BY v.NumVol, e.fonction;
```

(b) Nombre de vols associés à chaque pilote

```
SELECT e.nom, COUNT(v.NumVol) AS nb_vols
FROM VOL_OR v, TABLE(v.Equipage) e
WHERE e.fonction = 'Pilote'
GROUP BY e.nom;
```

(c) Impact de chaque indice par vol

Explication : $\text{impact} = \text{valeur} \times \text{poids}$ via `get_impact()`.

```
SELECT v.NumVol, i.nom_indice, i.get_impact() AS impact
FROM VOL_OR v, TABLE(v.IndicesQualite) i;
```

(d) Impact moyen par indice

```
SELECT i.nom_indice, AVG(i.get_impact()) AS impact_moyen
FROM VOL_OR v, TABLE(v.IndicesQualite) i
GROUP BY i.nom_indice;
```

3.3 Modèle logique

3.3.1 Prédicats extensionnels

Prédicats utilisés :

- vol(Dep, Arr, HDep, HArr)
- plusPetit(H1, H2)

3.3.2 Villes connectées (direct ou avec connexions)

Explication :

- règle de base = vol direct
- règle récursive = enchaînement compatible sur les horaires.

```
vol(paris, londres, 10, 11).
vol(londres, new_york, 14, 18).
vol(new_york, tokyo, 21, 25).
vol(tokyo, sydney, 28, 35).
vol(paris, rio, 23, 29).

plusPetit(11, 14).
plusPetit(18, 21).
plusPetit(25, 28).

trajet(D, A, HD, HA) :- vol(D, A, HD, HA).
trajet(D, A, HD, HA) :- vol(D, V, HD, H1), trajet(V, A, H2, HA), plusPetit(H1, H2).
```

Requête test :

```
trajet(paris, A, HD, HA)?
```

3.3.3 Villes connectées avec un nombre impair de connexions

Explication : alternance entre conn_pair et conn_impair.

```
conn_pair(D, A, HD, HA) :- vol(D, A, HD, HA).  
conn_pair(D, A, HD, HA) :- vol(D, V, HD, H1), conn_impair(V, A, H2, HA), plusPetit(H1, H2).  
conn_impair(D, A, HD, HA) :- vol(D, V, HD, H1), conn_pair(V, A, H2, HA), plusPetit(H1, H2).
```

Requête test :

```
conn_impair(paris, A, HD, HA)?
```

3.4 Modèle graphe

3.4.1 Modélisation

Explication :

- noeuds Aeroport
- arcs orientés VOL_VERS pour les vols directs.

3.4.2 Implémentation Neo4j

```
CREATE (p:Aeroport {id: 1, nom: 'Charles de Gaulle', ville: 'Paris', pays: 'FR'})  
CREATE (r:Aeroport {id: 2, nom: 'Antonio Carlos Jobim', ville: 'Rio', pays: 'BR'})  
CREATE (l:Aeroport {id: 3, nom: 'Heathrow', ville: 'Londres', pays: 'GB'})  
CREATE (ny:Aeroport {id: 4, nom: 'JFK', ville: 'New York', pays: 'US'})  
CREATE (t:Aeroport {id: 5, nom: 'Narita', ville: 'Tokyo', pays: 'JP'})  
CREATE (s:Aeroport {id: 6, nom: 'Kingsford Smith', ville: 'Sydney', pays: 'AU'})  
CREATE (a:Aeroport {id: 7, nom: 'Auckland Airport', ville: 'Auckland', pays: 'NZ'})  
CREATE (b:Aeroport {id: 8, nom: 'Brandenburg', ville: 'Berlin', pays: 'DE'})  
CREATE (ro:Aeroport {id: 9, nom: 'Fiumicino', ville: 'Rome', pays: 'IT'})  
  
CREATE (p)-[:VOL_VERS {compagnie: 'Air France', num_vol: 442}]->(r)  
CREATE (p)-[:VOL_VERS {compagnie: 'British Airways', num_vol: 101}]->(l)  
CREATE (l)-[:VOL_VERS {compagnie: 'Virgin Atlantic', num_vol: 202}]->(ny)  
CREATE (ny)-[:VOL_VERS {compagnie: 'JAL', num_vol: 303}]->(t)  
CREATE (t)-[:VOL_VERS {compagnie: 'Qantas', num_vol: 404}]->(s)  
CREATE (s)-[:VOL_VERS {compagnie: 'Air New Zealand', num_vol: 505}]->(a)  
CREATE (b)-[:VOL_VERS {compagnie: 'Lufthansa', num_vol: 606}]->(ro);
```

3.4.3 Requête demandée (paires de villes + distance, sans boucle)

```
MATCH chemin = shortestPath((depart:Aeroport)-[:VOL_VERS*]->(arrivee:Aeroport))
WHERE depart.ville <> arrivee.ville
RETURN depart.ville AS VilleDepart,
        arrivee.ville AS VilleArrivee,
        length(chemin) AS Distance
ORDER BY VilleDepart ASC, VilleArrivee ASC;
```